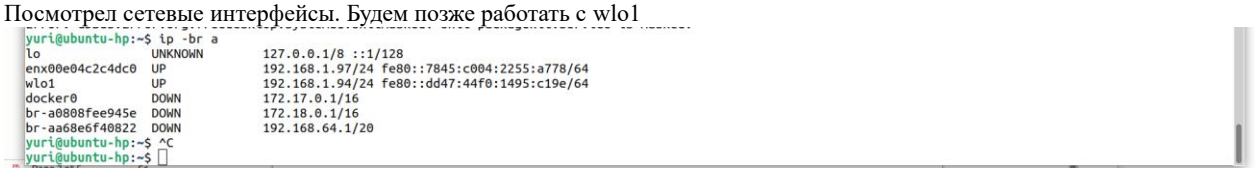
Домашнее задание 1. Сетевой стек

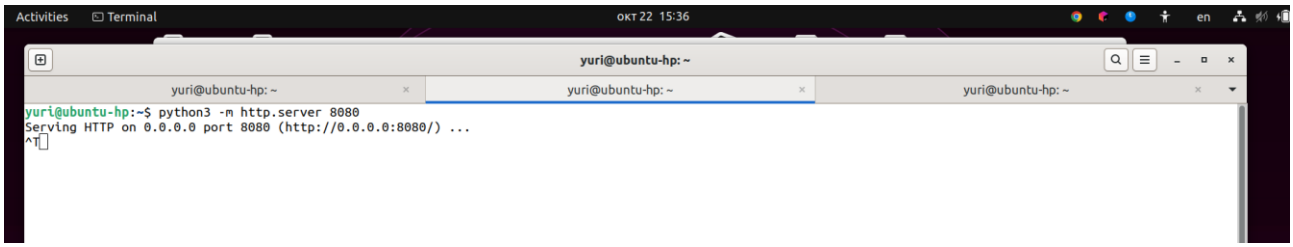
Выполнил Уфимцев Юрий

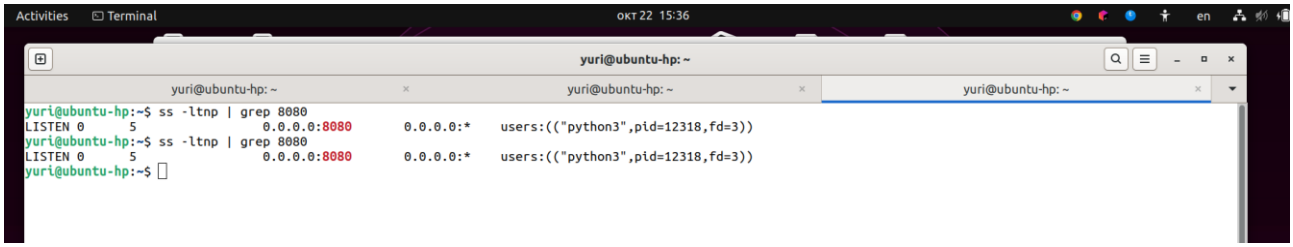
Сетевые интерфейсы

Посмотрел сетевые интерфейсы. Будем позже работать с wlo1

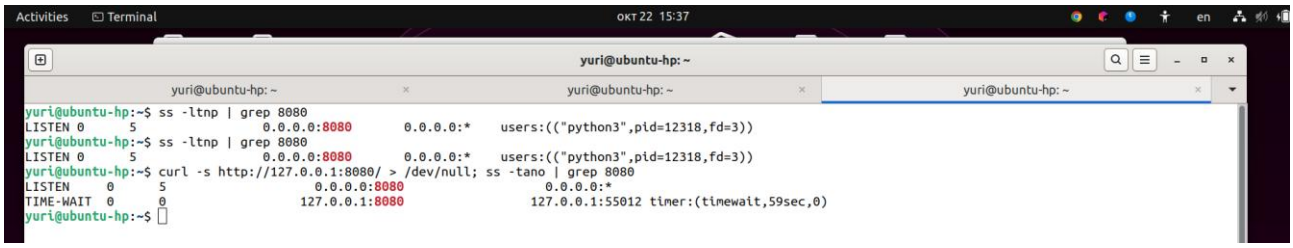


Задание 1. Анализ состояний TCP-соединений





Видим сокет в состоянии LISTEN на 0.0.0.0:8080 и процесс python3



В TIME-WAIT переходит сторона, которая первой закрыла соединение, Python http.server по умолчанию работает по HTTP/1.0 и закрывает соединение сразу после ответа. То есть инициатор закрытия сервер, и TIME-WAIT остаётся на стороне сервера, на порту 8080.

Сокет ждёт и решает две задачи:

1. Надёжное завершение. Если последний ACK потерялся, вторая сторона повторит FIN, и мы сможем ответить ACK, а не RST.
2. Защита нового соединения от старых пакетов. Опоздавшие сегменты прошлого соединения (IP:порт - IP:порт) не попадут в новое соединение.

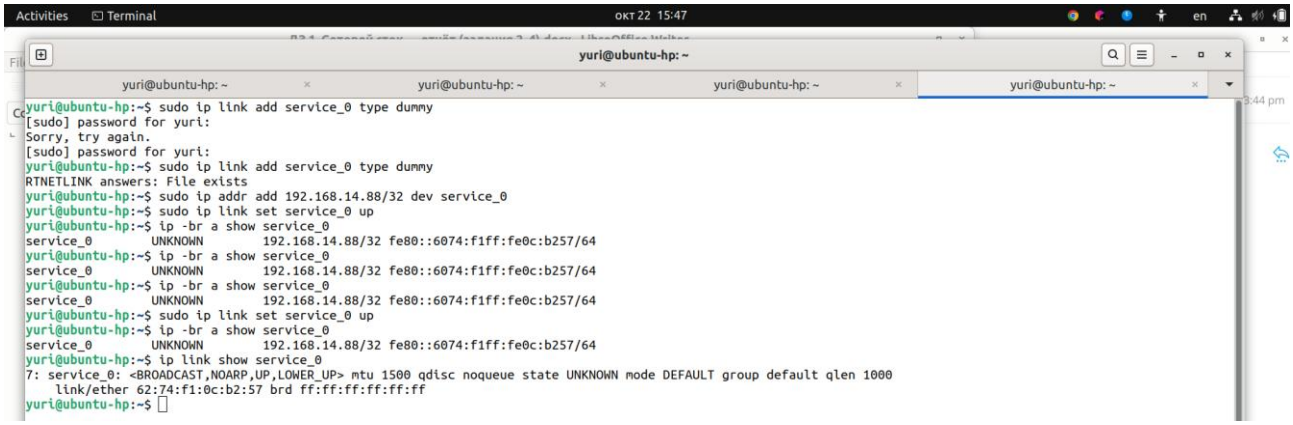
Приложение уже закрыло сокет, поэтому у него нет процесса и файлового дескриптора, нечего закрывать. Сокетом владеет ядро и удаляет его само по таймеру. Принудительное удаление сломало бы обе гарантии выше. Повлиять можно только через net.ipv4.tcp_tw_reuse и tcp_max_tw_buckets.

Проблемы от большого количества TIME-WAIT:

- Исчерпание эфемерных портов на клиенте (прокси, балансировщике) при множестве коротких соединений к одному адресу и порту. Диапазон ip_local_port_range около 28 тыс. портов на 60 секунд, то есть примерно 470 новых соединений в секунду. Дальше появится ошибка Cannot assign requested address.
- Расход памяти ядра и рост хеш-таблиц соединений.
- Переполнение tcp_max_tw_buckets: в логе появляется time wait bucket table overflow, и соединения закрываются без TIME-WAIT.

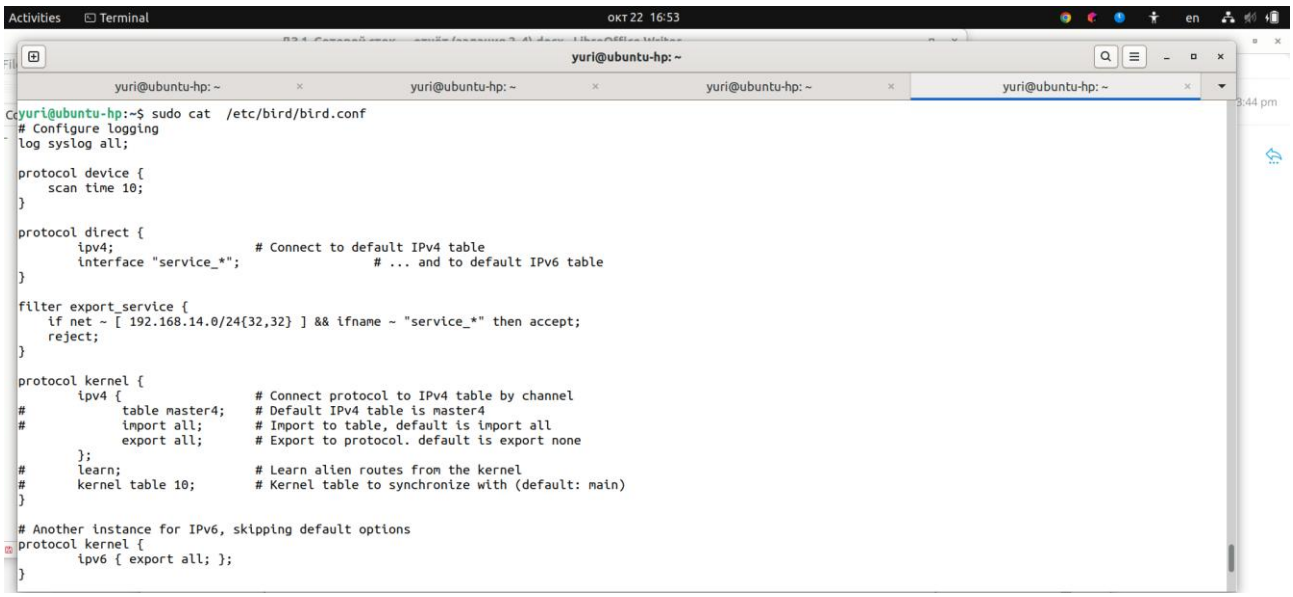
Решения: keep-alive и пулы соединений, tcp_tw_reuse=1, расширение диапазона портов, несколько исходящих IP-адресов.

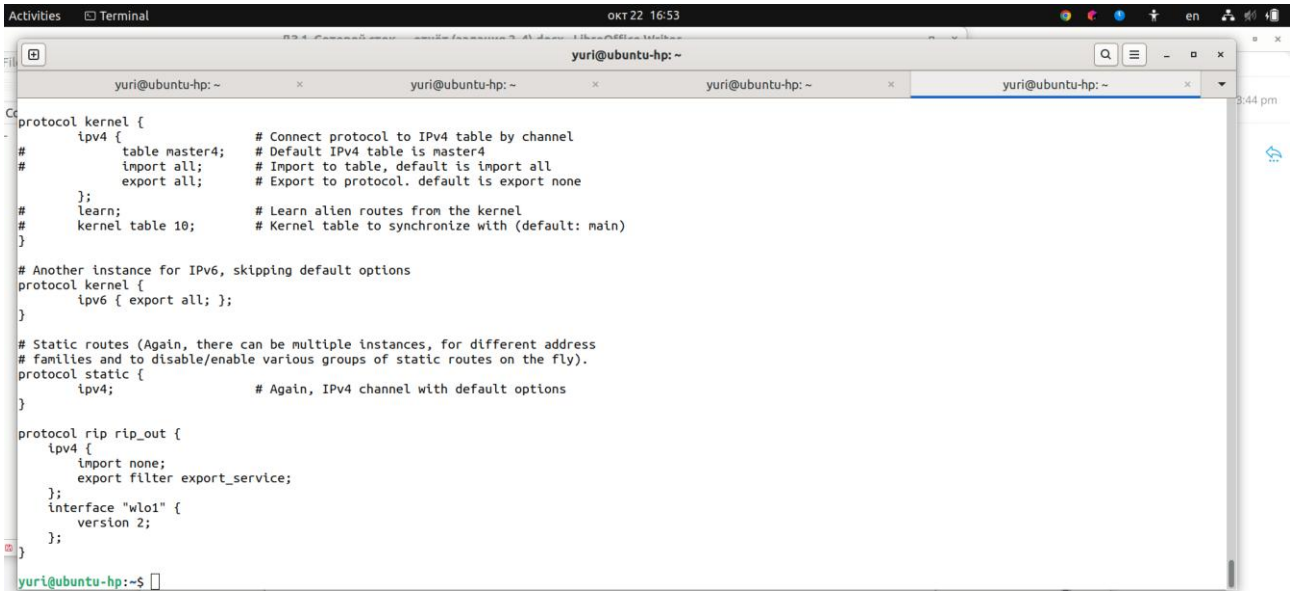
Задание 2. Динамическая маршрутизация с BIRD



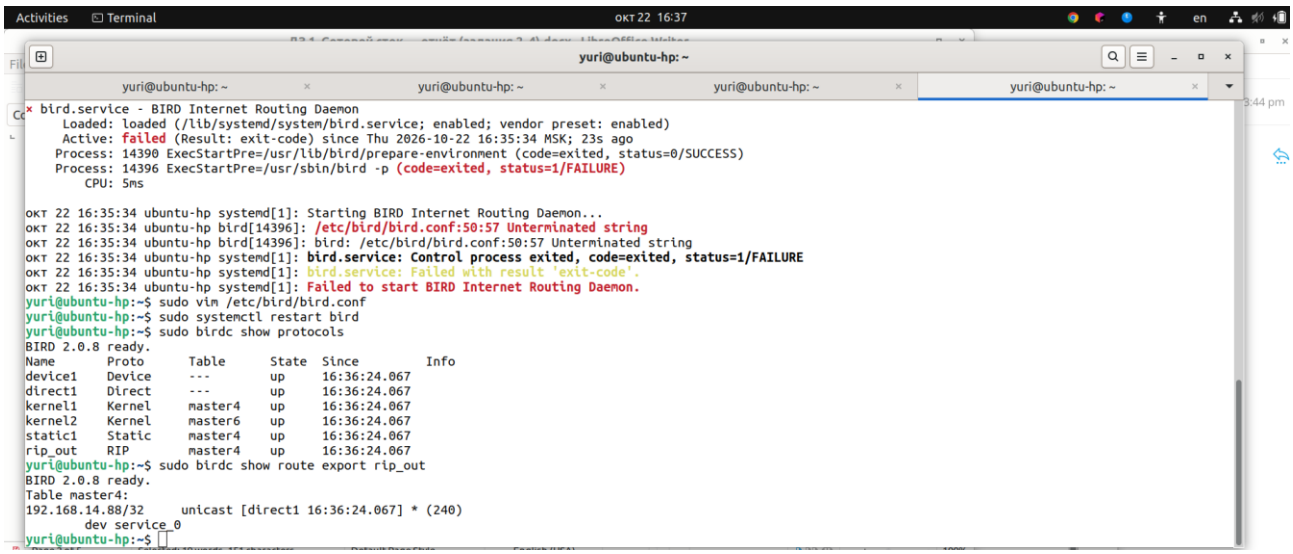
Создаём интерфейс service_0 с адресом 192.168.14.88/32. Работоспособность подтверждают флаги UP,LOWER_UP в выводе ip link show service_0

Настраиваем bird

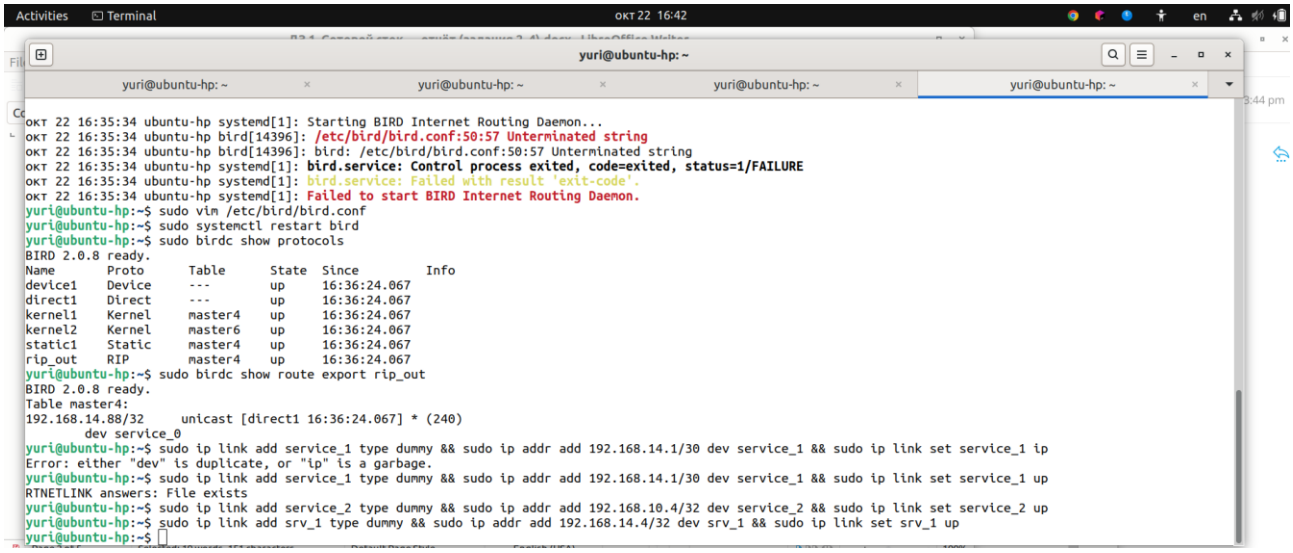




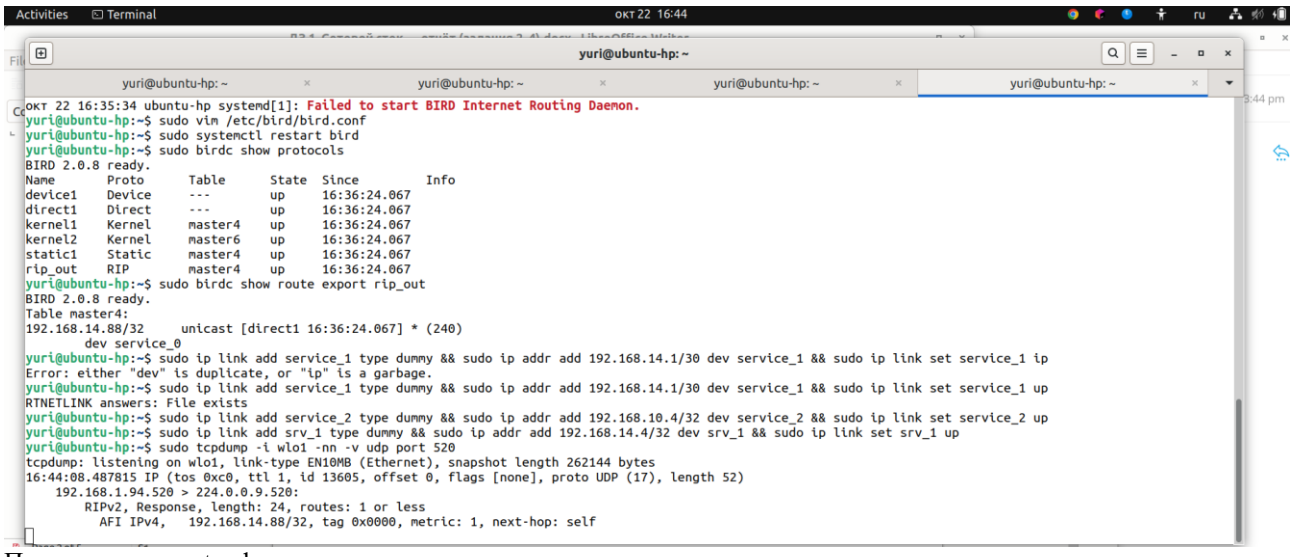
- protocol device - BIRD отслеживает появление и изменение интерфейсов, поэтому будущие интерфейсы service_* подхватываются автоматически.
- protocol direct - импортирует в таблицу BIRD подключённые сети, только с интерфейсов, чьё имя начинается на service_.
- filter export_service - пропускает только префиксы из 192.168.14.0/24 с маской /32}) и только с интерфейсов service_*.
- protocol rip - RIP version 2 на основном интерфейсе, экспорт через фильтр



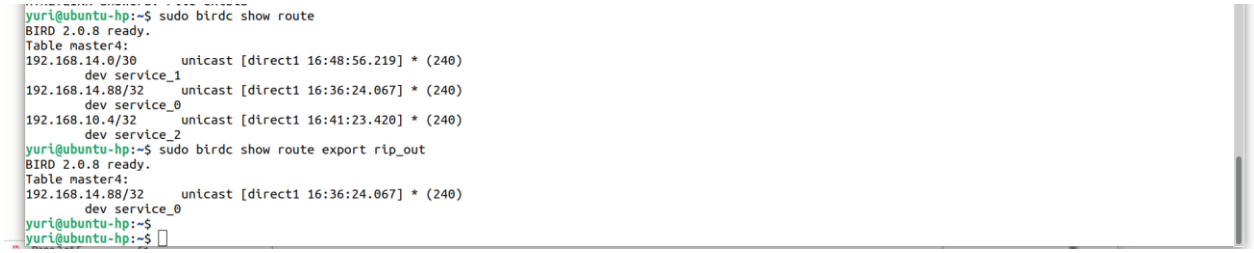
Протоколы в состоянии up, на экспорт 192.168.14.88/32



Создаём интерфейсы



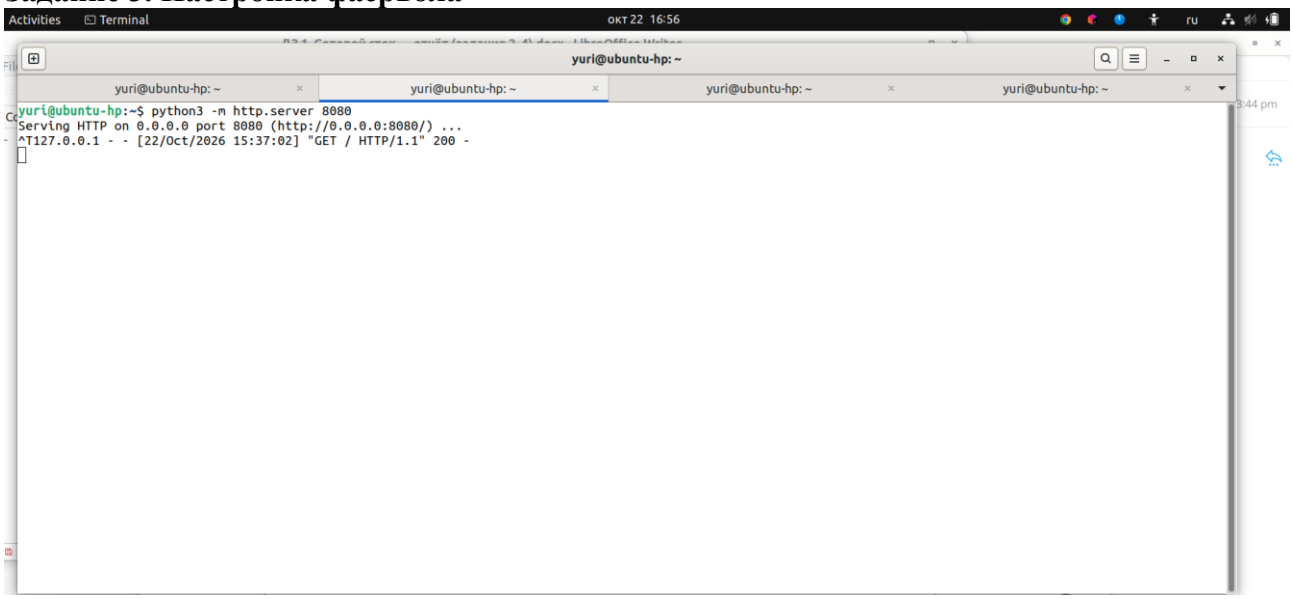
Проверяем через tcpdump



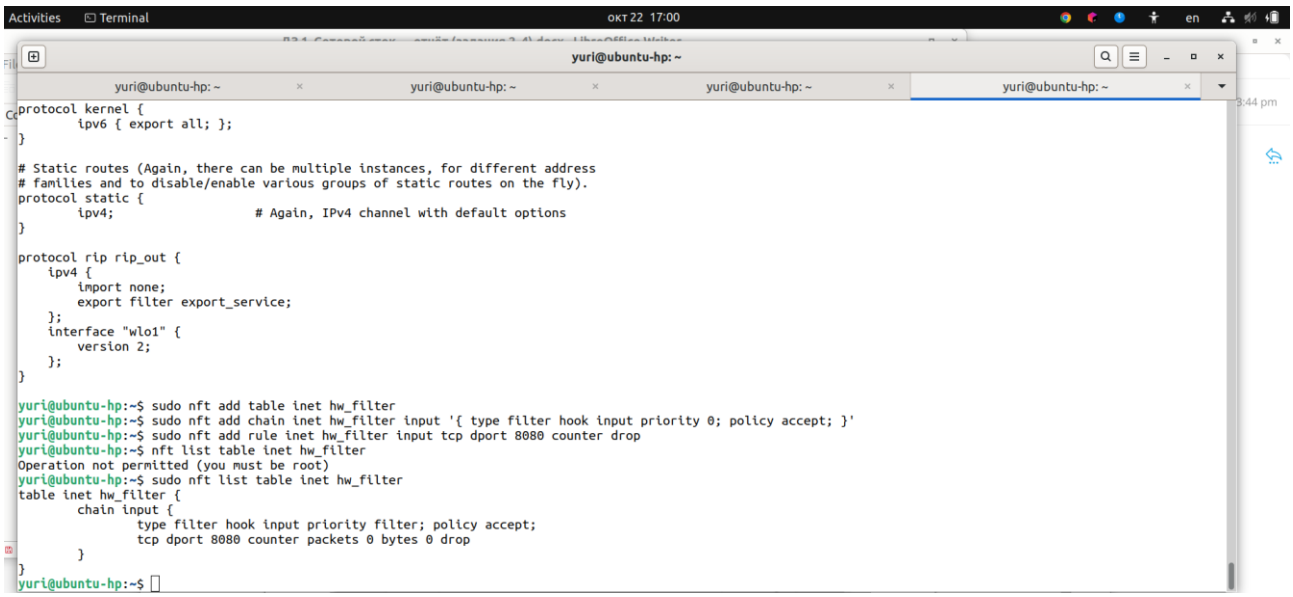
В пакетах видим только `192.168.14.88/32`, успешно.
При этом видим, что в таблице BIRD есть `192.168.14.0/30` и `192.168.10.4/32`, но они не анонсируются.

- service_1 не анонсируется, потому что у него маска /30, а не /32.
- service_2 не анонсируется, потому что его адрес не из подсети 192.168.14.0/24.
- srv_1 не анонсируется, потому что его имя не начинается на service_.

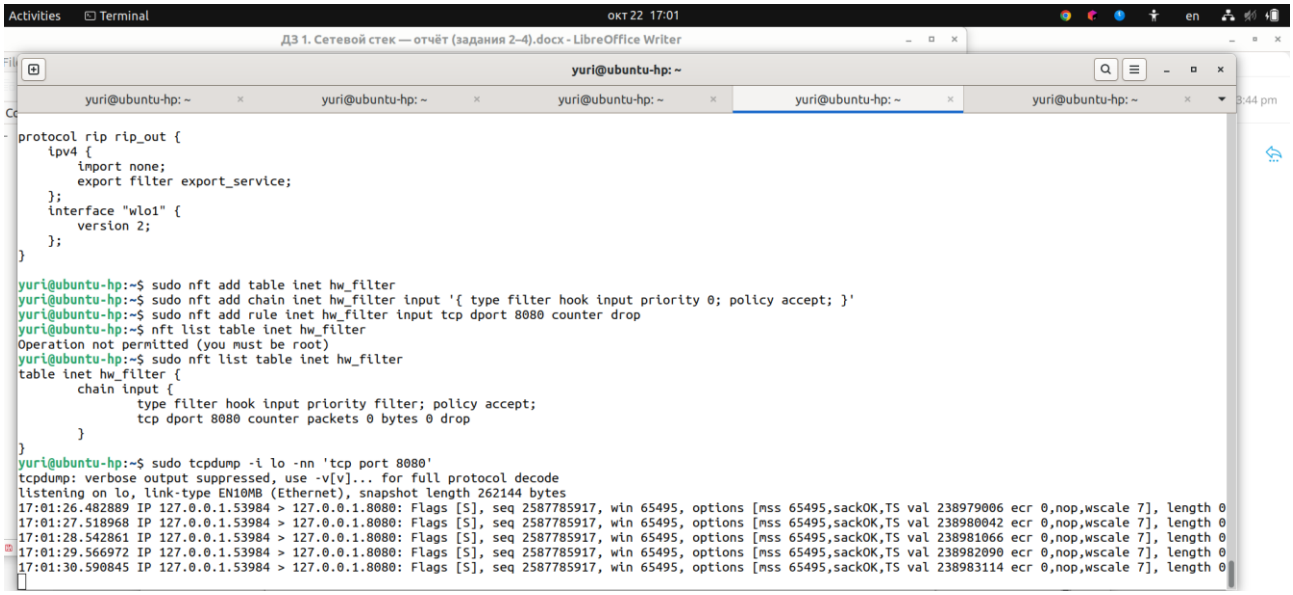
Задание 3. Настройка фаервола



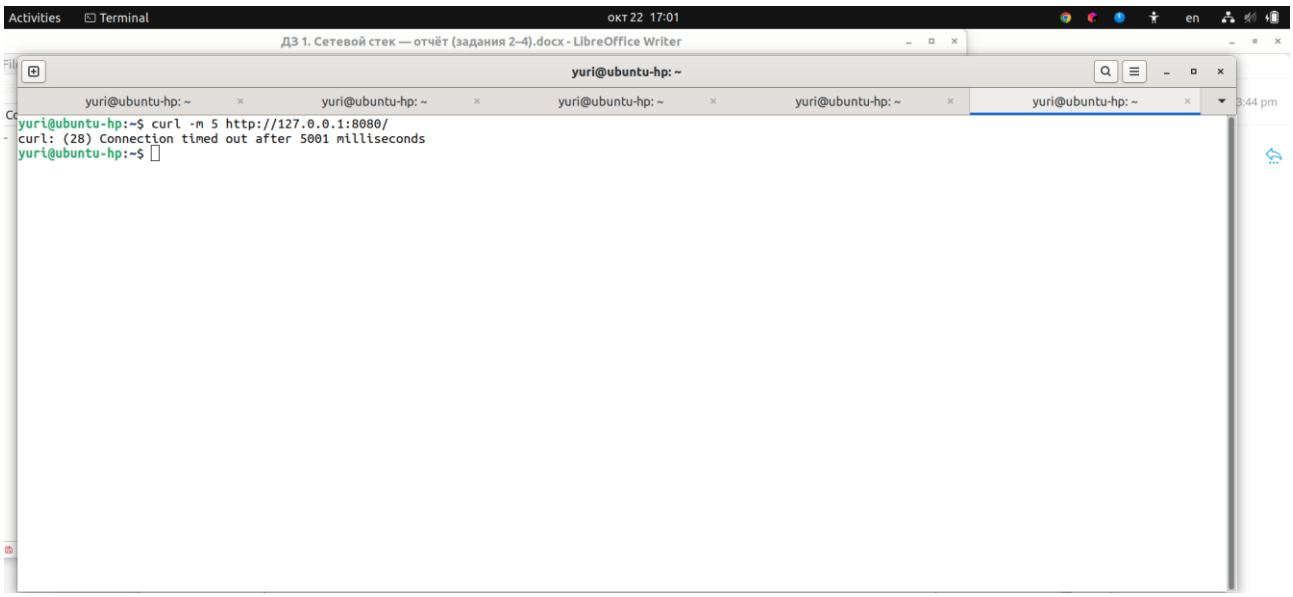
Веб-сервер из задания 1 запущен

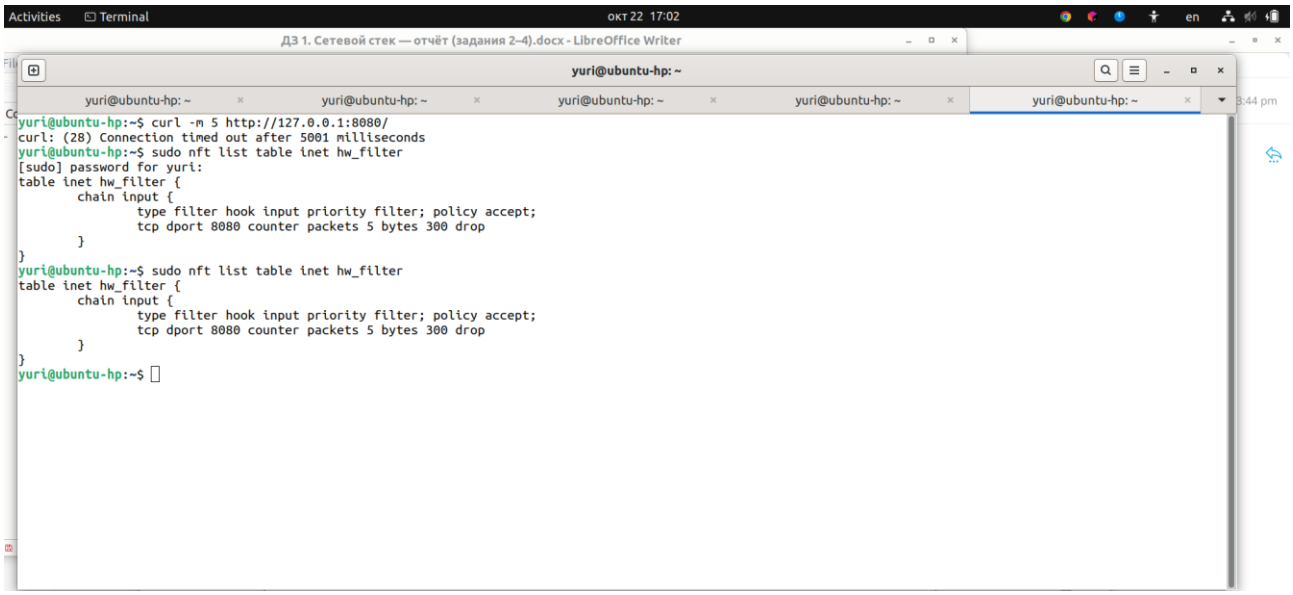


Создаём правило



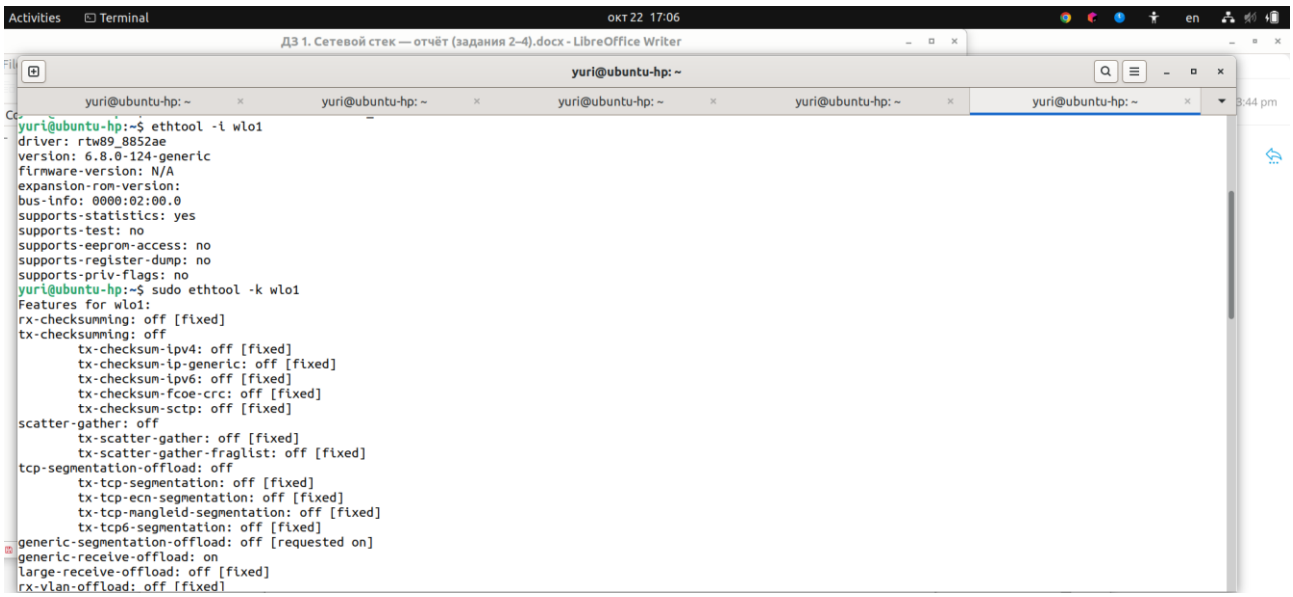
curl падает по таймауту. В tcpdump видим только повторные SYN (Flags [S]) без SYN-ACK: пакеты доходят до интерфейса, но фаервол их отбрасывает.

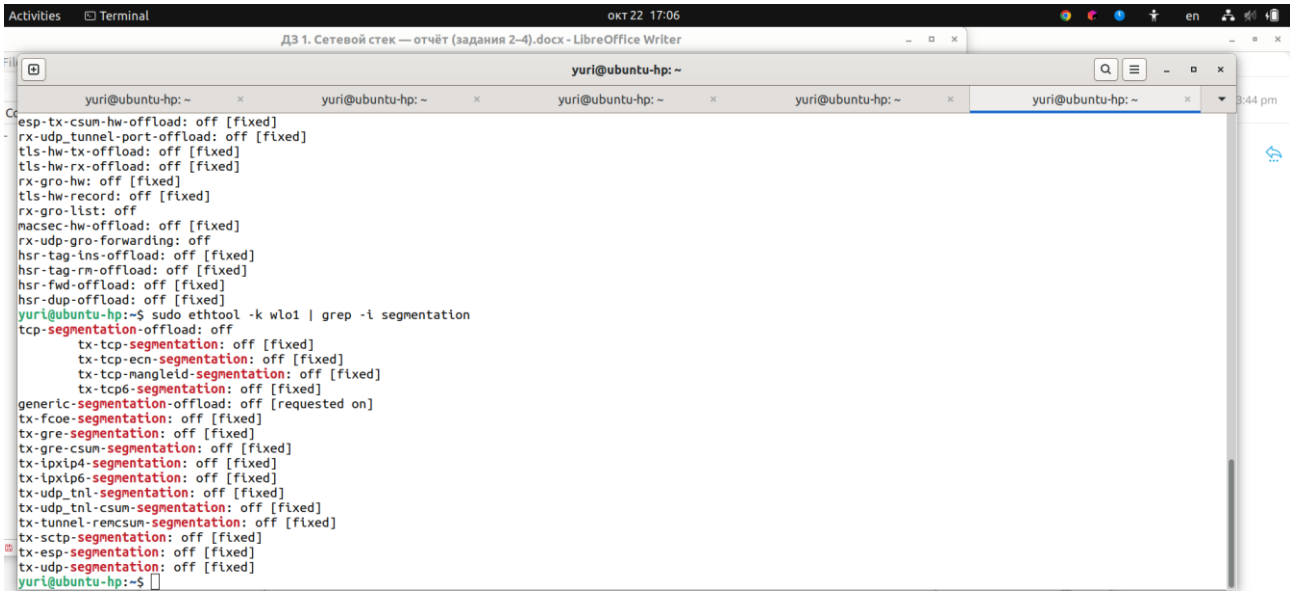




Также видим, что счетчик counter packets увеличился

Задание 4. Offloading





TCP Segmentation Offload (TSO) переносит нарезку TCP-данных на сегменты с CPU на сетевую карту. Ядро отдаёт адаптеру один большой блок данных (до ~64 КБ) с шаблоном заголовков. Карта сама режет его на сегменты размером MSS, проставляет sequence numbers и IP ID и считает контрольные суммы.

В результате стек обрабатывает один «большой пакет» вместо десятков маленьких. То есть меньше нагрузки на CPU, меньше дескрипторов и прерываний и выше пропускная способность.

TSO требует включённых checksum offload и scatter-gather. Если аппаратного TSO нет, ядро использует программный аналог GSO. На приёме работают аналогичные механизмы GRO/LRO.

Побочный эффект заключается в том, что tcpdump на отправителе может показывать пакеты больше MTU, потому что он видит данные до нарезки картой.